请详细描述 MySQL 的 B+ 树中查询数据的全过程 - 面试鸭 | 2026最新面试题+详细答案解析

> ## Excerpt
>
> 2026最新后端、MySQL、数据库面试题，包含详细答案解析和练习题，26532+程序员已学习，已帮助10万+程序员通过校招/社招/实习面试，拿到offer！

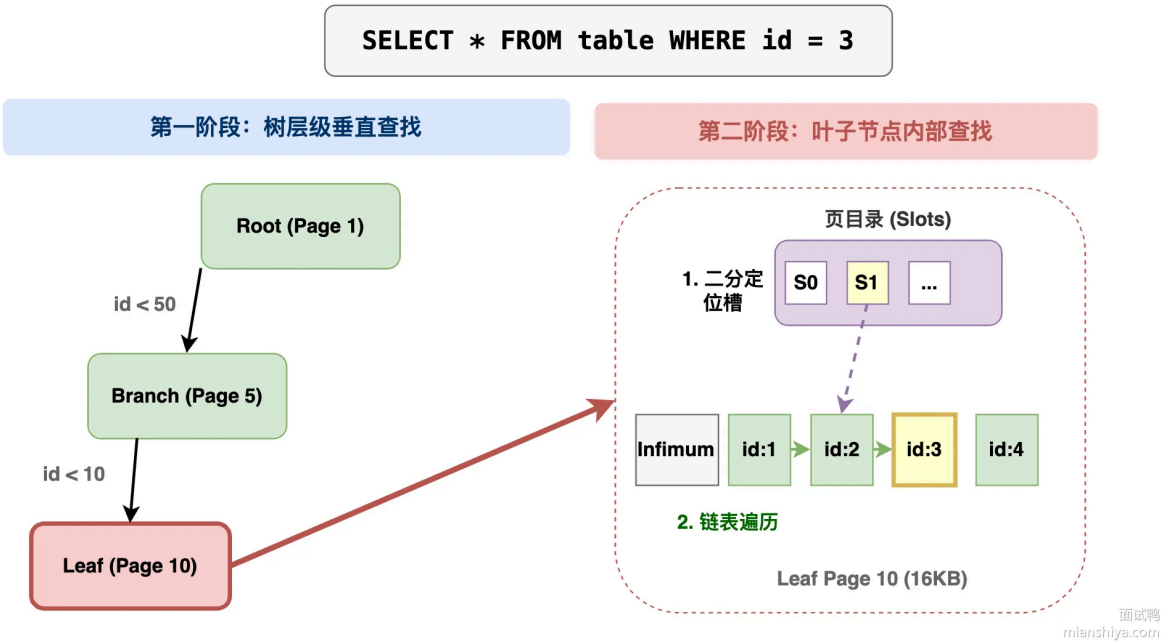
回答重点

B+ 树查询数据分两个阶段：先从根节点往下定位到叶子节点，再在叶子节点内部定位到具体的数据行。

第一阶段是**树的垂直查找**。从根节点开始，把查询的键值跟节点中存储的索引键值做比较，用二分法确定落在哪个区间，然后顺着指针往下走到子节点。这个过程一直重复，直到到达叶子节点。一棵 3 层的 B+ 树最多只需要 3 次磁盘 I/O 就能定位到叶子节点。

第二阶段是**页内查找**。叶子节点是一个 16KB 的数据页，里面存了很多条记录。InnoDB 用**页目录**来加速查找，页目录把记录分成若干个组，每个槽指向一个组的最大记录。先用二分法在页目录里定位到记录所在的组，再沿着组内的单向链表遍历，找到目标记录。

整个过程就是：根节点 → 中间节点 → 叶子节点 → 页目录二分 → 组内链表遍历。



扩展知识

叶子节点的内部结构

叶子节点不是简单地把记录堆在一起，内部有精心设计的结构。

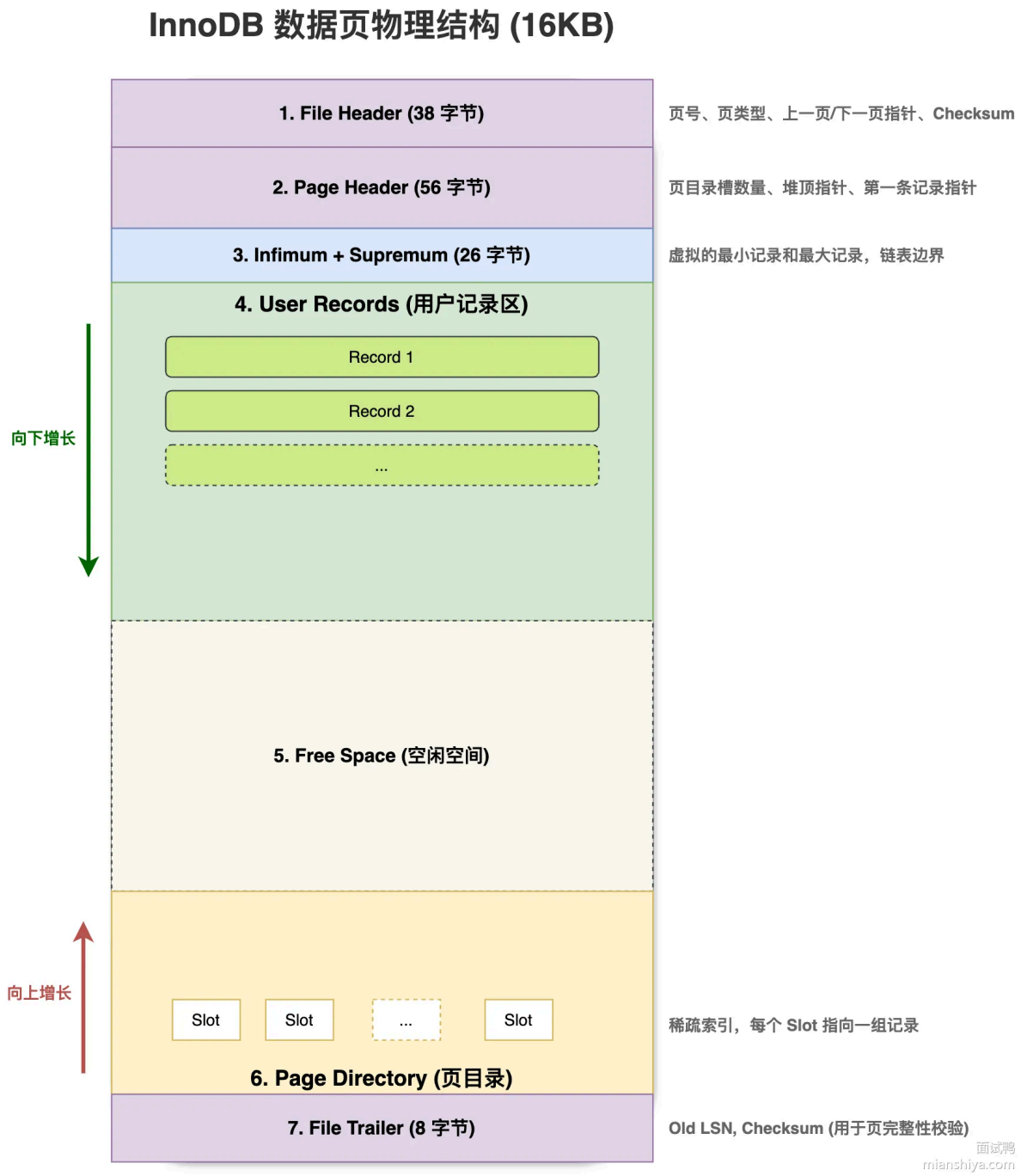
一个 16KB 的数据页大概长这样：

1）**文件头和页头**：记录页的基本信息，比如页号、页类型、上一页和下一页的指针。叶子节点之间通过双向链表串起来，范围查询的时候可以沿着链表往后扫。

2）**用户记录区**：实际存储数据行的地方。每条记录按主键顺序排列，记录之间用单向链表串联。

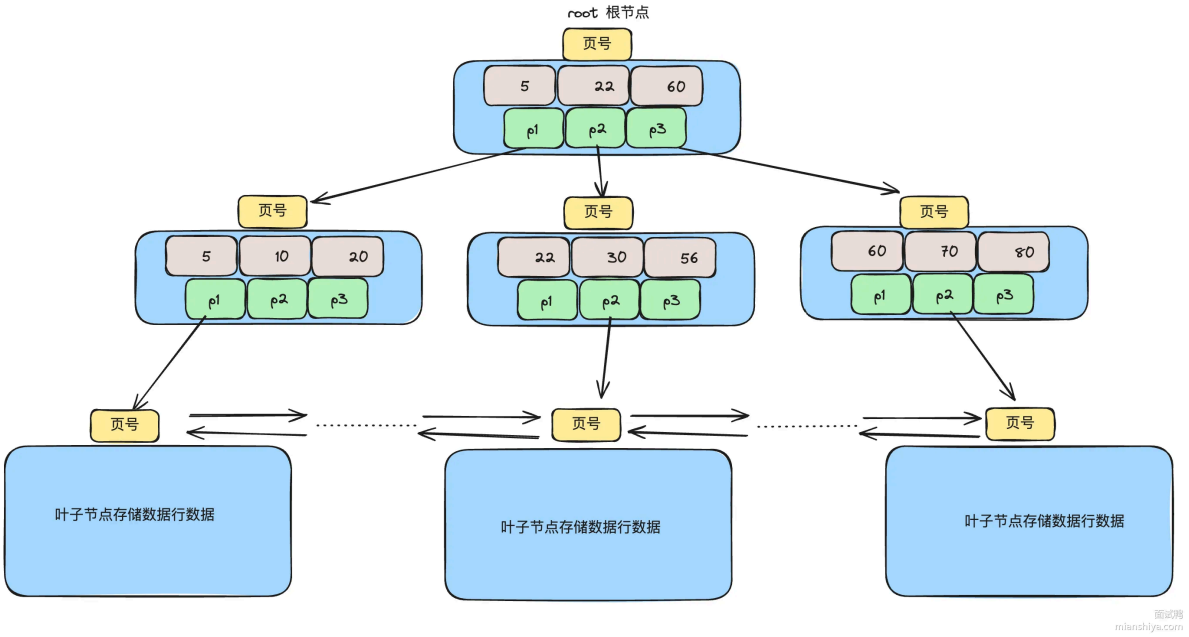
3）**页目录**：这是加速查找的关键。InnoDB 把记录分成若干组，每组 4-8 条记录，页目录里的每个槽指向对应组的最大记录。查找时先对槽做二分，快速缩小范围，再在组内遍历链表。

4）**文件尾**：用于校验页的完整性，存了 checksum。

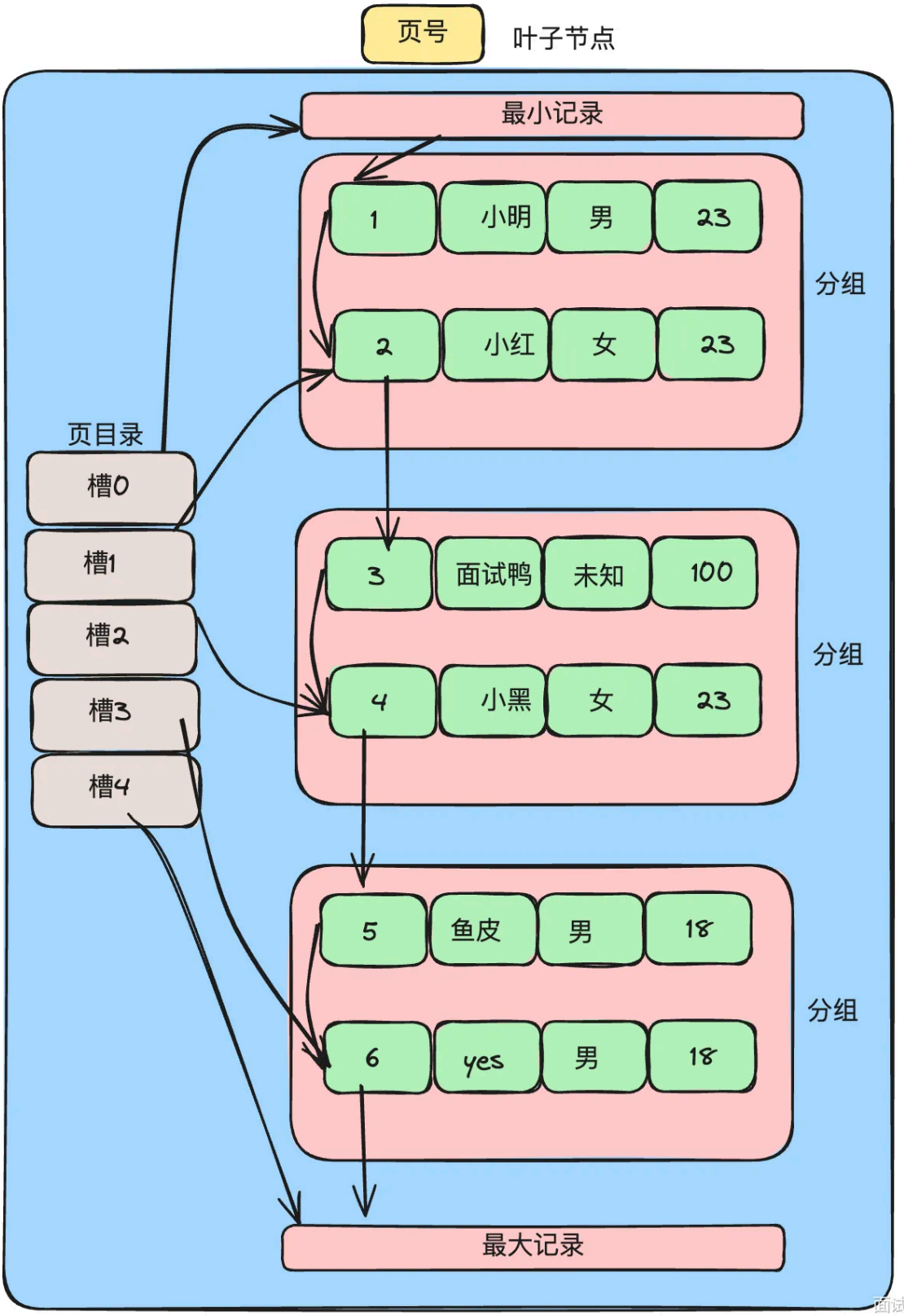


页目录二分查找的细节

数据从根节点找起，根据比较数据键值与节点中存储的索引键值（非叶子节点也有页目录，也能用二分查找，详细看下面叶子节点的分析），确定数据落在哪个区间，从而确定分支，从上到下最终定位到叶子节点。



定位到叶子节点后，因为一片叶子默认有 16KB 大小，所以理论上可以存多条记录。叶子节点的实际构造如下图所示：



面试鸭
mianshiya.com

从上图可以知晓，叶子节点有**页目录**结构（非叶子节点也有，当前拿叶子节点分析，结构是一样的），它其实就是一个索引，通过它可以快速找到记录。

页目录分为了多个槽，每个槽都指向对应一个分组内的最大记录，每个分组内都会包含若干条记录。

通过**二分查询**，利用槽就能直接定位到记录所在的组，从而就能获取到对应的记录。

举个例子，现在有 5 个槽，如果想查找主键为 3 的记录，此时的流程是：

1）通过二分得到槽的中间位置，`low = 0`，`high = 4`，`(0+4)/2 = 2`；2）通过槽定位到第二个分组中的主键为 4 的记录，4 大于 3，`low = 0` 不变，`high = 2`；3）继续二分 `(0+2)/2 = 1`；槽 1 中主键 2 小于 3，`low = 1`，`high = 2`；4）此时 `high - low = 1`，可以确定值在 high 即槽 2 中，但是槽 2 只能定位到主键为 4 的记录，又因为槽之间是挨着的，所以可以得到槽 1 的位置，从槽 1 入手拿到 主键 2 的记录，然后因为记录是通过单向链表串起来的，往下遍历即可定位到主键 3 的记录。

以上就是利用二分查询的定位流程。通过槽可找到对应记录所在的组，或能直接定位到记录，或还需通过链表遍历找到对应的数据。

实际上，每个分组的记录数是有规定的，图中做了省略只画了两条，InnoDB 规定：

- 第一个分组只有一条记录
- 中间的分组 4-8 条记录
- 最后一个分组 1-8 条记录

因此不必担心遍历很长的链表导致性能问题。

这题的重点是先简单提下从根节点遍历到子节点的过程，然后提到每页节点默认大小为 16KB ，所以理论上能存储很多记录，从而引出页目录，再通过二分查找才能对应记录。

Buffer Pool 的作用

实际查询时不是每次都从磁盘读，InnoDB 有个 Buffer Pool 会缓存热点数据页。查询流程实际是：

1）先在 Buffer Pool 里找目标页，命中就直接用，不用走磁盘

2）没命中就从磁盘读页到 Buffer Pool，再返回数据

3）根节点和热点的中间节点基本常驻 Buffer Pool，大部分查询只需要 1 次磁盘 I/O 读叶子节点

所以说 B+ 树查询只需要 3 次 I/O 是理论上限，实际上因为 Buffer Pool 的存在，平均 I/O 次数要低得多。

面试官追问

提问：为什么 B+ 树比 B 树更适合做数据库索引？

> 详细解答可以看：[为什么 MySQL 选择使用 B+ 树作为索引结构？]

回答：两个核心原因。第一，B+ 树的非叶子节点不存数据，只存索引键和指针，一个页能容纳更多条目，树更矮，I/O 次数更少。第二，B+ 树的叶子节点之间有双向链表，范围查询时可以顺着链表扫，不用回溯到父节点；B 树的范围查询要中序遍历整棵树，I/O 是随机的，慢很多。

提问：一个 3 层的 B+ 树大概能存多少数据？

> 详细解答可以看：[MySQL 三层 B+ 树能存多少数据？]

回答：假设主键是 bigint 8 字节，指针 6 字节，一个非叶子节点能存约 1170 个条目。叶子节点假设每行数据 1KB，一页能存 16 条。那么 3 层 B+ 树：第一层 1 个根节点，第二层 1170 个中间节点，第三层 1170 × 1170 ≈ 137 万个叶子节点，每个叶子 16 条数据，总共能存约 2000 万条记录。

提问：聚簇索引和二级索引的查询过程有什么区别？

> 详细解答可以看：[MySQL InnoDB 引擎中的聚簇索引和非聚簇索引有什么区别？]

回答：聚簇索引的叶子节点直接存完整的数据行，查到叶子节点就拿到数据了。二级索引的叶子节点存的是索引列的值加主键值，查到之后还要拿着主键值去聚簇索引再查一遍，这叫回表。所以二级索引查询最多要走两棵 B+ 树，聚簇索引查询只走一棵。

提问：页目录为什么不把每条记录都建一个槽？

回答：空间成本太高。每个槽占 2 字节，如果每条记录一个槽，一页存 1000 条记录就要 2KB 的页目录空间，占整页的 12.5%。分组的方式，每组 4-8 条记录，1000 条记录只需要 125-250 个槽，页目录只占几百字节。用组内链表遍历换空间，遍历最多 8 条，代价可以接受。